

This program is a **simple LangChain chatbot with memory**. Its purpose is to show how a model can remember earlier conversation turns by storing chat history and automatically injecting that history into later prompts.

What this program does overall

It:

1. Installs the required LangChain libraries.
2. Creates a chat model.
3. Builds a prompt that includes past conversation history.
4. Creates a memory store for each user session.
5. Wraps the chain with a history manager.
6. Runs two turns in the same session.
7. Shows that the model can answer “What is my name?” because it remembers “My name is Alice.”

The key difference from the no-memory version is simple:
this program **stores and reuses earlier messages**.

Step-by-step explanation

1) Installing the packages

The first line installs:

- langchain
- langchain-openai
- langchain-core

Why this matters:

- langchain gives you the chaining framework
- langchain-openai connects LangChain to OpenAI chat models
- langchain-core provides prompt, message, and runnable components

In Colab, this step is necessary because those libraries may not already be installed.

2) Importing the modules

The program imports:

- os
- defaultdict from collections
- ChatOpenAI

- ChatPromptTemplate
- MessagesPlaceholder
- InMemoryChatMessageHistory
- RunnableWithMessageHistory

What each one is doing:

- os: used to read the API key from environment variables
- defaultdict: automatically creates a new memory object when a new session ID appears
- ChatOpenAI: the model interface
- ChatPromptTemplate: used to structure the prompt
- MessagesPlaceholder: a slot where previous chat history will be inserted
- InMemoryChatMessageHistory: stores messages in Python memory
- RunnableWithMessageHistory: wraps a chain so history is automatically read and written

This set of imports tells you the program has two goals:

- talk to the model
- manage conversation history

3) Checking the API key

The program checks whether OPENAI_API_KEY exists.

If the key is missing, it raises an error immediately.

Why this is useful:

- it fails early
- it gives a clear message
- it avoids a more confusing authentication failure later

So this part is just a safety check before making model calls.

4) Creating the model

The model is created with:

- model = gpt-4o-mini
- temperature = 0

What that means:

- gpt-4o-mini is the selected chat model

- temperature 0 makes responses more consistent and less random

Why that matters in a demo:

You want stable behavior so the memory effect is easy to observe. If temperature were high, the replies might vary more and distract from the main lesson.

5) Building the prompt

The prompt has three parts:

1. A system message
“You are a helpful assistant. Keep answers short.”
2. A history placeholder
This is where past messages will be inserted.
3. The current human input
This is the new user message for the current turn.

This is the most important design change compared with the no-memory example.

Why MessagesPlaceholder matters

Without it, the prompt only contains the current message.

With it, the prompt can include prior turns such as:

- user: Hi, my name is Alice.
- assistant: Nice to meet you, Alice.

That allows the model to answer follow-up questions in context.

6) Creating the chain

The chain connects:

- the prompt
- the model

So the flow becomes:

current input + prior history → formatted prompt → model response

By itself, this chain still does not know where history comes from. It only knows that the prompt has a place for history. The actual history management is added in the next steps.

7) Creating the memory store

The program creates a variable called store using defaultdict with InMemoryChatMessageHistory.

What this means:

- store acts like a dictionary

- each session ID gets its own chat history
- if a session ID does not exist yet, a new empty history is created automatically

Why defaultdict is smart here

Normally, if you ask for a dictionary key that does not exist, you get an error.

With defaultdict, asking for a new session ID automatically creates a fresh memory object.

So:

- `store["demo-user"]` gives one conversation history
- `store["another-user"]` would give a different conversation history

This is how the program separates one user's memory from another's.

8) The `get_session_history` function

This function takes a `session_id` and returns the matching history object from store.

Its job is very simple:

- receive a session ID
- look up that session's chat history
- return it

Why it exists:

`RunnableWithMessageHistory` needs a function that knows how to fetch the right history for a given session. This function is that bridge.

9) Wrapping the chain with memory

This is the core of the program.

`RunnableWithMessageHistory` wraps the original chain and adds automatic history handling.

It is configured with:

- the base chain
- the function that fetches history
- `input_messages_key = "input"`
- `history_messages_key = "history"`

What these settings mean

`input_messages_key = "input"`

This tells LangChain that the current user message comes in under the key named `input`.

So when you call invoke with:
input = "Hi, my name is Alice."

LangChain knows that is the new human message.

history_messages_key = "history"

This tells LangChain that the prompt expects past messages under the variable name history.

That matches the MessagesPlaceholder in the prompt.

Why this wrapper is powerful

It automatically does three things:

1. reads the existing history for the session
2. inserts that history into the prompt
3. stores the new user and assistant messages after the call

So you do not need to manually append messages every turn.

10) Defining the session

The config contains a session ID:
demo-user

This tells the program which memory bucket to use.

Why this matters:

Memory is tied to the session ID, not just the notebook.

If you used:

- demo-user for turn 1
- demo-user for turn 2

then both turns share the same history.

If you changed the second one to:

- another-user

then the model would act like it was a brand-new conversation.

Practical takeaway

The session ID is what makes the conversation persistent across turns within the running app.

11) Turn 1: introducing Alice

In the first invoke call, the input is:
"Hi, my name is Alice."

Here is what happens internally:

1. LangChain checks config and sees session_id = demo-user
2. It fetches that session's history from store
3. Since this is the first turn, the history is empty
4. The prompt is built with:
 - system message
 - no prior history yet
 - current user message
5. The model generates a response
6. LangChain stores both:
 - the user message
 - the assistant response

So after turn 1, memory now contains the first exchange.

12) Turn 2: asking “What is my name?”

In the second invoke call, the input is:

“What is my name?”

Now the internal behavior is different:

1. LangChain again uses session_id = demo-user
2. It fetches the existing history for that session
3. This time the history already contains turn 1
4. It builds a prompt with:
 - system message
 - prior conversation history
 - current user question
5. The model sees the earlier message containing “my name is Alice”
6. It can answer correctly

This is why the memory version works while the no-memory version may fail.

13) Why the model now remembers

The model is not magically remembering on its own.

The program is remembering for it.

That distinction matters.

Important concept

LLMs are still generally stateless between API calls.

What this program does is:

- save prior messages externally
- resend them as context on the next call

So the “memory” is really an application-level memory layer.

The intelligence comes from the model.

The continuity comes from the stored history.

14) Printing the stored history

At the end, the program loops through:

```
store["demo-user"].messages
```

This prints each saved message and its type.

You will typically see a sequence like:

- HumanMessage: Hi, my name is Alice.
- AIMessage: Nice to meet you, Alice.
- HumanMessage: What is my name?
- AIMessage: Your name is Alice.

Why this is useful:

It proves the memory actually exists and shows exactly what has been stored.

This is a good debugging step because you can inspect whether the conversation history is being built correctly.

15) Why this is better than the no-memory version

The no-memory version sends each message independently.

This version sends each message with prior context.

That means this program can support:

- follow-up questions
- conversational continuity
- personalization within a session

- multi-turn chatbot behavior

So if the first program demonstrates the problem, this program demonstrates the fix.

16) What “in-memory” means

The class used here is `InMemoryChatMessageHistory`.

That means the chat history is stored only in the notebook’s live Python memory.

Benefits

- very simple
- great for demos
- easy to understand
- no database required

Limitations

- memory disappears if the Colab runtime restarts
- memory is not shared across separate machines or deployments
- not suitable by itself for production chat applications

So this is ideal for learning, but not enough for a real enterprise chatbot.

17) Hidden trade-off: memory grows over time

Each turn adds more messages to history.

That is good for continuity, but over time it creates issues:

- larger prompts
- higher token usage
- increased cost
- slower responses
- more irrelevant context

In real applications, teams usually add:

- summarization
- windowed memory
- retrieval
- persistence layers

This example intentionally keeps things simple so the core idea is easy to see.

18) Mental model for this program

Think of the app as keeping a notebook for each user session.

For each session:

- it writes down what the user said
- it writes down what the assistant replied
- before answering the next question, it rereads those notes

That is what this code is doing.

19) What each major component is responsible for

A clean way to understand the design is by role:

- ChatOpenAI
Generates the response
- ChatPromptTemplate
Defines the structure of the messages sent to the model
- MessagesPlaceholder
Creates a slot for past chat history
- InMemoryChatMessageHistory
Stores prior messages
- defaultdict
Creates separate histories for separate sessions automatically
- get_session_history
Retrieves the correct history object
- RunnableWithMessageHistory
Connects the chain to the history system
- session_id in config
Chooses which conversation memory to use

That is the whole architecture in one view.

20) Bottom line

This program is a **session-based conversational LangChain example with memory**.

Its real lesson is not just “the bot remembers,” but **how** it remembers:

- the app stores prior messages
- the app retrieves them by session ID
- the app injects them into the next prompt

- the model answers using that accumulated context

So the memory is not automatic model memory. It is **managed conversational state** built around the model.

Short summary

If you need one-line understanding:

This program works because it saves each conversation turn in a session-specific history and reuses that history in later prompts, allowing the chatbot to answer follow-up questions consistently.